EXPERIMENT 8: DESIGN AND TRAIN A CONVOLUTIONAL NEURAL NETWORK (CNN) ON IMAGE DATASETS (MNIST)

1. DATASET SOURCE

The dataset used in this experiment is the **MNIST Handwritten Digits Dataset**.

- **Source Link:** https://www.kaggle.com/datasets/oddrationale/mnist-in-csv

2. DATASET DESCRIPTION

The **MNIST dataset** is one of the most widely used benchmark datasets in computer vision and deep learning.

General Characteristics

- **Domain:** Image Classification
- **Type:** Supervised Learning Dataset
- **Total Samples:** 70,000 images
  - Training Set: 60,000 images
  - Test Set: 10,000 images
- **Image Size:** 28 × 28 pixels
- **Color Format:** Grayscale (single channel)
- **Pixel Value Range:** 0–255 (normalized to 0–1 during preprocessing)

Features

Each image is represented as a **28×28 matrix of pixel intensities**, flattened into a vector of 784 values when required.

Target Variable

- Integer labels from **0 to 9**
- Represents the digit shown in the image

Key Properties

- Balanced dataset (roughly equal samples per class)
- Low noise and well-centered digits
- Ideal for testing classification algorithms, especially CNNs

3. MATHEMATICAL FORMULATION OF THE ALGORITHM (CNN)

A **Convolutional Neural Network (CNN)** is a deep learning model specifically designed for processing grid-like data such as images.

(a) Convolution Operation

The core operation in CNN is convolution:

$$S(i,j) = (X * K)(i,j) = \sum_m \sum_n X(i+m, j+n) \cdot K(m,n)$$

Where:

- (X): Input image
- (K): Kernel (filter)
- (S): Feature map

(b) Activation Function (ReLU)

$$f(x) = \max(0, x)$$

Introduces non-linearity and helps the network learn complex patterns.

(c) Pooling Layer (Max Pooling)

$$P(i,j) = \max_{(m,n)\in \text{window}} X(i+m, j+n)$$

Reduces spatial dimensions while preserving important features.

(d) Fully Connected Layer

$$y = W \cdot x + b$$

Where:

- (W): Weight matrix
- (x): Input vector
- (b): Bias

(e) Softmax Function (Output Layer)

$$P(y=i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

Converts outputs into probabilities for classification.

(f) Loss Function (Categorical Cross-Entropy)

$$L = -\sum_{i=1}^{10} y_i \log(\hat{y}_i)$$

Measures the difference between predicted and actual labels.

4. ALGORITHM LIMITATIONS

Despite its effectiveness, CNN has several limitations:

(a) Data Dependency

- Requires a **large amount of labeled data**
- Performance degrades with insufficient training samples

(b) Computational Complexity

- High computational cost due to:
  - Multiple layers
  - Large number of parameters
- Requires GPU for efficient training

(c) Overfitting

- May memorize training data if:
  - Model is too complex
  - Regularization is not applied

(d) Lack of Interpretability

- CNN acts as a **black-box model**
- Difficult to understand decision-making process

(e) Sensitivity to Transformations

- Performance may degrade under:
  - Rotation
  - Scaling
  - Translation (unless augmented)

5. METHODOLOGY / WORKFLOW

The complete workflow followed in this experiment is as follows:

Step 1: Data Loading

- Load MNIST dataset using Keras/TensorFlow
- Split into training and testing sets

Step 2: Data Preprocessing

- Normalize pixel values:

$$X = \frac{X}{255}$$

- Reshape data to include channel dimension:

$$(28,28) \rightarrow (28,28,1)$$

- Convert labels into **one-hot encoding**

Step 3: Model Architecture Design

Typical CNN architecture used:

1. **Convolution Layer (Conv2D)**
2. **Activation (ReLU)**
3. **Pooling Layer (MaxPooling2D)**
4. Repeat Conv + Pool layers
5. **Flatten Layer**
6. **Dense (Fully Connected Layer)**
7. **Output Layer (Softmax)**

Step 4: Model Compilation

- Optimizer: **Adam**
- Loss Function: **Categorical Crossentropy**
- Metrics: **Accuracy**

Step 5: Model Training

- Train model using:
  - Epochs (e.g., 10–20)
  - Batch size (e.g., 32 or 64)
- Backpropagation used to update weights

Step 6: Model Evaluation

- Evaluate performance on test dataset
- Calculate:
  - Accuracy
  - Loss

Step 7: Prediction

- Use trained model to predict digits on unseen data

6. PERFORMANCE ANALYSIS

Evaluation Metrics Used

(a) Accuracy

(b) Loss

- Indicates how well the model fits the data
- Lower loss → better performance

Observed Results (Typical for MNIST CNN)

- Training Accuracy: ~99%
- Test Accuracy: ~98–99%
- Loss decreases steadily across epochs

Interpretation

- High accuracy indicates **strong generalization**
- Small gap between training and test accuracy → minimal overfitting
- CNN effectively captures spatial features like edges and shapes

7. HYPERPARAMETER TUNING

Hyperparameter tuning plays a crucial role in improving model performance.

Parameters Tuned

(a) Number of Filters

- Tested values: 32, 64, 128
- More filters → better feature extraction but higher cost

(b) Kernel Size

- Common choice: 3×3
- Larger kernels increase computation without significant gain

(c) Batch Size

- Tested: 32, 64, 128
- Smaller batch size → better generalization but slower training

(d) Learning Rate

- Controlled by optimizer (Adam default: 0.001)
- Too high → unstable training
- Too low → slow convergence

(e) Number of Epochs

- Increasing epochs improves accuracy up to a limit
- Too many epochs → overfitting

(f) Dropout Rate (Regularization)

- Typical value: 0.25–0.5
- Helps reduce overfitting

Conclusion

This experiment demonstrates the effectiveness of **Convolutional Neural Networks** in image classification tasks. The CNN model successfully learns hierarchical features from raw pixel data and achieves high accuracy on the MNIST dataset. Proper preprocessing, architecture design, and hyperparameter tuning significantly enhance performance.